Agora — Product Requirements Document

Version: 1.0 **Last Updated**: 2026-04-13 **Status**: Draft **Author**: Xingfan Xia

1. Executive Summary

Agora is an open-source, general-purpose multi-agent collaboration platform where users create rooms, populate them with AI agents backed by different LLM models (Claude, GPT, Gemini, Qwen, etc.), and watch them interact through structured or freeform conversation. Games — roundtable debates, werewolf, murder mystery, TRPG — are the initial wedge because the space is wide open (no existing project has gained meaningful traction) and the format is inherently viral. But the platform architecture is designed from day one to support any multi-agent interaction mode: company simulations, brainstorming sessions, educational scenarios, customer service practice, and user-defined custom modes. The core insight is that good multi-agent collaboration and good game experience require the same underlying capabilities: persona management, information isolation, structured decision-making, flow control, and memory.

2. Problem Statement

The Pain

Multi-agent AI interaction is one of the most compelling capabilities unlocked by modern LLMs, yet there is no general-purpose platform for orchestrating it. Today, anyone who wants multiple AI agents to interact must:

- Write custom orchestration code from scratch for each scenario

- Manually manage prompt routing, turn order, and information visibility
- Build their own UI for observing multi-agent conversations
- Handle model-specific API differences themselves
- Reinvent memory, persona, and flow control for every project

The result is that multi-agent interaction remains locked behind engineering effort. The viral "Three Departments and Six Ministries" demo, AI roundtable debates, and LLM-powered werewolf games all demonstrate massive user interest — but each is a one-off hack with no reusable infrastructure.

Why Now

1. **LLM quality has crossed the threshold.** Models from multiple providers (Claude, GPT-4o, Gemini 2.5, Qwen 3) can now reliably follow complex persona instructions, maintain character consistency, and produce structured output. Multi-agent interaction is no longer a research curiosity — it works.
2. **Structured output is production-ready.** Zod-constrained generation (via Vercel AI SDK) means agents can make verifiable decisions (votes, choices, game actions) without hallucination.
3. **The market is empty.** No open-source multi-agent collaboration platform has gained traction. Projects in this space have fewer than ~100 GitHub stars. First-mover advantage is real.
4. **Viral distribution is built in.** AI agents arguing, playing games, and surprising each other produce shareable moments. Every session is potential content.

3. Target Users

Phase 1 — Initial Users (Months 1-3)

User Type	Description	Primary Need
AI Enthusiasts	People who follow AI Twitter/Reddit, experiment with prompts, compare models	Watch different models interact; see which model "wins" debates or games
Content Creators	YouTubers, streamers, bloggers who cover AI	Generate shareable AI interaction content with minimal setup
Developers	Engineers exploring multi-agent architectures	Understand multi-agent patterns; fork and extend for their own projects

Phase 2 — Growth Users (Months 3-9)

User Type	Description	Primary Need
Tabletop/Board Game Fans	People who play werewolf, murder mystery, TRPG	Play with AI agents when friends aren't available; novel game experiences
Educators	Teachers and trainers	Create practice scenarios (debates, negotiations, role-play exercises)
Researchers	People studying LLM behavior, alignment, emergent dynamics	Controlled multi-agent experiments with different models

Phase 3 — Platform Users (Months 9+)

User Type	Description	Primary Need
Mode Creators	Developers who build custom interaction modes	Platform to distribute and monetize custom multi-agent experiences
Enterprise Teams	Companies doing AI-assisted brainstorming, simulation	Private deployment of multi-agent collaboration for business scenarios

4. Product Vision & Goals

Vision Statement

Agora is the default platform people reach for when they want multiple AI agents to interact — whether for entertainment, education, research, or work.

Short Term (0-3 months)

- Ship a working roundtable debate mode that demonstrates the platform's core value
- Achieve 500+ GitHub stars through viral demos and developer interest
- Validate that the platform core (Agent, Room, Channel, Flow) is genuinely mode-agnostic by shipping a second mode (Werewolf) without modifying core

Mid Term (3-9 months)

- Ship 4 interaction modes (Roundtable, Werewolf, Script Kill, TRPG)
- Support human-AI hybrid sessions (humans participate alongside AI agents)
- Build spectator and replay features that enable content creation
- Reach 2,000+ GitHub stars and an active contributor community

Long Term (9-18 months)

- Enable user-created custom modes via a published Mode SDK
- Launch an agent marketplace (shareable personas, pre-built teams)
- Support hierarchical multi-agent workflows (manager agents coordinating worker agents)
- Establish Agora as the standard infrastructure for multi-agent interaction

5. Core Platform Capabilities

These capabilities are mode-agnostic. They form the foundation (`packages/core`) that all interaction modes build on.

5.1 Agent System

An Agent is the fundamental unit. It wraps an LLM with a persona, model configuration, and a clean two-method interface borrowed from AgentScope's `AgentBase`.

Data Model

```
interface AgentConfig {
  id: string
  name: string
  persona: string              // Short description (1-3 sentences from
  enrichedPersona?: string     // AI-generated full character (auto-enri
  modelProvider: ModelProvider  // 'anthropic' | 'openai' | 'google' | '
  modelId: string              // 'claude-sonnet-4-20250514' | 'gpt-4o'
  systemPrompt?: string        // Override (normally constructed from pe
  temperature?: number         // Default 0.7
  avatar?: string              // URL or generated
  metadata?: Record<string, unknown>
}
```

Core Interface

```
interface Agent {
  readonly config: AgentConfig

  /**
   * Given observations (messages the agent can see), produce a reply.
   * The mode determines WHAT the agent replies to; the agent determines
   */
  reply(observations: Message[], context: AgentContext): Promise<AgentRe

  /**
   * Passively observe messages without replying.
   * Used for accumulating context (e.g., watching a debate before your
   */
  observe(messages: Message[]): void
}
```

Agent Lifecycle

1. **Creation**: User provides name + short persona + model selection
2. **Enrichment**: Platform calls an LLM to expand the short persona into a full character description with personality traits, speech patterns, knowledge domains, and behavioral tendencies
3. **Initialization**: Agent receives mode-specific system prompt (persona + mode rules + current state)
4. **Active**: Agent participates in the room via `reply()` and `observe()`
5. **Suspended**: Agent is temporarily inactive (e.g., eliminated in werewolf)
6. **Terminated**: Session ends; agent state is archived

Persona Enrichment

When a user writes "A sarcastic philosophy professor who loves Nietzsche," the platform auto-enriches this into a detailed character sheet:

- **Personality traits**: Sardonic wit, intellectual elitism tempered by genuine love of ideas
- **Speech patterns**: Uses rhetorical questions, drops Latin phrases, references obscure philosophers

- **Knowledge domains**: Continental philosophy, existentialism, ethics
- **Behavioral tendencies**: Challenges weak arguments aggressively, respects well-reasoned positions even if they disagree, quotes Nietzsche at every opportunity

The user can review and edit the enriched persona before the session starts.

Model Selection

Each agent can use a different LLM model. The platform abstracts model differences via Vercel AI SDK:

Provider	Models	Strengths
Anthropic	Claude Sonnet 4, Claude Opus 4	Strong reasoning, character consistency
OpenAI	GPT-4o, o3	Fast, good at structured output
Google	Gemini 2.5 Pro, Gemini 2.5 Flash	Multimodal, large context
Alibaba	Qwen Max, Qwen Plus	Chinese language, cost-effective

5.2 Room System

A Room is the container for a multi-agent interaction session.

Data Model

```
interface RoomConfig {
  id: string
  name: string
  mode: ModeId                      // 'roundtable' | 'werewolf' | 'scrip
  modeConfig: Record<string, unknown>  // Mode-specific settings
  agents: AgentConfig[]
  humanParticipants?: HumanParticipant[]
  maxAgents: number                 // Default 12
  visibility: 'public' | 'private'
  createdBy: string
  status: 'configuring' | 'active' | 'paused' | 'completed'
}
```

Room Lifecycle

1. **Configuration**: Creator selects mode, adds agents, configures settings
2. **Ready Check**: All agents validated, mode requirements met (e.g., werewolf needs minimum players)
3. **Active**: Interaction is running; flow controller manages turns
4. **Paused**: Interaction suspended (can resume)
5. **Completed**: Interaction finished; results archived, replay available

Room Operations

- `createRoom(config)` : Initialize a new room
- `addAgent(roomId, agentConfig)` : Add an agent to a room
- `removeAgent(roomId, agentId)` : Remove an agent
- `startRoom(roomId)` : Begin the interaction
- `pauseRoom(roomId)` : Suspend the interaction
- `resumeRoom(roomId)` : Resume from pause
- `endRoom(roomId)` : End the interaction and archive

5.3 Channel System

Channels control information isolation — who can see what. This is the mechanism that makes games like werewolf possible (wolves have a private channel; dead players can't see living player discussions) and also enables private brainstorming, manager-only communications, and any scenario where not all participants should see all messages.

Inspired by AgentScope's MsgHub pattern.

Data Model

```
interface Channel {
  id: string
  roomId: string
  name: string
  type: ChannelType
  members: string[]          // Agent IDs
  visibility: ChannelVisibility
  metadata?: Record<string, unknown>
}

type ChannelType =
  | 'public'       // All agents can see (town square in werewolf)
  | 'private'      // Only members can see (wolf chat)
  | 'broadcast'    // One-to-many, no replies (GM announcements)
  | 'whisper'      // 1:1 ephemeral (seer checking a player)
  | 'spectator'    // Observers only, agents can't see

type ChannelVisibility =
  | 'members_only'    // Only channel members see messages
  | 'spectator_visible' // Members + spectators can see
  | 'god_mode'         // Only god-mode observers see (internal agent re
```

Channel Operations

- `createChannel(config)` : Create a new channel in a room
- `addMember(channelId, agentId)` : Add an agent to a channel
- `removeMember(channelId, agentId)` : Remove an agent from a channel
- `postMessage(channelId, message)` : Send a message to a channel

- `getVisibleMessages(agentId)`: Get all messages an agent is allowed to see (across all channels they belong to)

Information Isolation Guarantees

Critical invariant: An agent's `reply()` method NEVER receives messages from channels it does not belong to. This is enforced at the platform level, not the mode level. No mode can accidentally leak information.

5.4 Flow Control System

Flow controllers determine who speaks when. Different interaction modes need different flow patterns.

FlowController Interface

```
interface FlowController {
  type: FlowType

  /**
   * Determine which agent(s) should act next.
   * Returns one or more agent IDs.
   */
  getNextActors(state: RoomState): string[]

  /**
   * Process an agent's action and advance the flow state.
   */
  advance(action: AgentAction, state: RoomState): FlowState

  /**
   * Check if the current flow phase/round is complete.
   */
  isPhaseComplete(state: RoomState): boolean

  /**
   * Check if the entire interaction is complete.
   */
  isComplete(state: RoomState): boolean
}
```

Built-in Flow Types

Flow Type	Description	Used By
FreeForm	Any agent can speak at any time; turn order determined by an orchestrator agent or by response timing	Free chat, brainstorming
RoundRobin	Agents speak in fixed or randomized order; each gets one turn per round	Roundtable debate
StateMachine	Flow moves through defined phases (e.g., Night → Day → Vote → Elimination); each phase has its own rules for who acts	Werewolf, Script Kill
Hierarchical	A "manager" agent delegates to "worker" agents; workers report back; manager synthesizes	Company simulation, complex workflows

FreeForm Details

FreeForm flow uses a lightweight orchestrator (either a dedicated agent or a simple algorithm) to decide who speaks next. The orchestrator considers:

- Who was most recently @mentioned
- Who hasn't spoken in a while
- Who has the most relevant context to contribute
- Random variation to prevent predictable patterns

This prevents the "everyone talks at once" problem while maintaining organic conversation flow.

5.5 Memory System

Agents need memory at two levels: within a session and across sessions.

Session Memory (Short-Term)

```
interface SessionMemory {
  /** All messages the agent has observed in this session */
  observations: Message[]

  /** Summarized context when observations exceed context window */
  summary?: string

  /** Mode-specific state (e.g., "I know Player 3 is a wolf") */
  modeState: Record<string, unknown>

  /** The agent's internal reasoning (not visible to other agents) */
  internalThoughts: string[]
}
```

Session memory is managed automatically: 1. All observed messages are stored 2. When observations approach the model's context limit, older messages are summarized 3. The agent always has access to: full recent context + summarized older context + mode state

Long-Term Memory (Cross-Session)

```
interface LongTermMemory {
  agentId: string

  /** Key facts the agent has learned */
  facts: Fact[]

  /** Relationships with other agents */
  relationships: Relationship[]

  /** Behavioral patterns (learned preferences, strategies) */
  patterns: Pattern[]
}
```

Long-term memory is opt-in (configured per mode). When enabled: - After each session, key facts and relationship changes are extracted and stored - In subsequent sessions, the agent starts with its accumulated knowledge - Useful for ongoing campaigns (TRPG) or persistent characters

5.6 Structured Output System

All agent decisions that affect game/interaction state must use Zod-constrained structured output. This prevents hallucination and ensures the platform can reliably parse agent actions.

Examples

```
// Werewolf vote
const VoteSchema = z.object({
  action: z.literal('vote'),
  target: z.string().describe('The agent ID to vote for elimination'),
  reasoning: z.string().describe('Why you are voting for this player (vi
})

// Debate position
const DebatePositionSchema = z.object({
  stance: z.enum(['strongly_agree', 'agree', 'neutral', 'disagree', 'str
  argument: z.string().describe('Your argument in 2-4 sentences'),
  rebuttal_to: z.string().optional().describe('Agent ID whose argument y
  concession: z.string().optional().describe('Any point you concede from
})

// TRPG action
const TRPGActionSchema = z.object({
  action_type: z.enum(['move', 'attack', 'cast_spell', 'use_item', 'talk
  description: z.string().describe('Narrative description of what you do
  target: z.string().optional(),
  dice_required: z.boolean().describe('Does this action require a dice r
})
```

Structured Output Flow

1. Mode defines the Zod schema for each decision point
2. Agent receives the schema as part of its prompt context
3. Vercel AI SDK's `generateObject()` constrains the LLM output to match the schema
4. Platform validates the output and applies it to game/interaction state
5. If validation fails, agent is re-prompted (max 3 retries)

5.7 Observation Layer

The observation layer enables spectating, replay, and analytics without interfering with the interaction.

Event Stream

Every action in the system emits an event:

```
interface AgoraEvent {
  id: string
  roomId: string
  timestamp: number
  type: EventType
  channelId?: string
  agentId?: string
  payload: unknown
  visibility: 'public' | 'spectator' | 'god_mode'
}

type EventType =
  | 'message'           // Agent sent a message
  | 'action'            // Agent took a structured action (vote, move, e
  | 'phase_change'      // Flow controller advanced to a new phase
  | 'agent_status'      // Agent status changed (active, suspended, elim
  | 'system'            // System event (room started, paused, ended)
  | 'internal_thought'  // Agent's internal reasoning (god_mode only)
```

Spectator Mode

Spectators see all `public` and `spectator_visible` events in real time via Socket.io. They cannot interact with agents or influence the session.

God Mode

God-mode observers see everything, including: - Private channel messages - Agent internal reasoning - Structured output before it's processed - Flow controller state transitions

This is primarily for debugging, research, and content creation.

Replay System

All events are persisted to Postgres. A completed session can be replayed: - Full replay at original speed or accelerated - Step-by-step replay with ability to pause and inspect state - Filtered replay (e.g., only show wolf channel during werewolf)

6. Mode System

Modes are pluggable interaction patterns built on top of the platform core. Each mode is a package that implements the `Mode` interface.

Mode Interface

```
interface Mode {
  /** Unique identifier */
  id: ModeId

  /** Display name */
  name: string

  /** Description for users */
  description: string

  /** Minimum and maximum agents */
  agentLimits: { min: number; max: number }

  /** Whether human participants are supported */
  supportsHumans: boolean

  /** Configuration schema (rendered as a form in the UI) */
  configSchema: z.ZodSchema

  /** Set up channels, assign roles, initialize state */
  initialize(room: Room, config: ModeConfig): Promise<ModeState>

  /** Create the flow controller for this mode */
  createFlowController(config: ModeConfig): FlowController

  /** Build the system prompt for an agent given its role and the curren
  buildAgentPrompt(agent: Agent, role: string, state: ModeState): string

  /** Define the structured output schemas for each decision point */
  getActionSchemas(phase: string): Record<string, z.ZodSchema>

  /** Process a structured action and update mode state */
  processAction(action: AgentAction, state: ModeState): ModeState

  /** Determine if the mode has reached a terminal state */
  isComplete(state: ModeState): boolean

  /** Generate the final summary/results */
  summarize(state: ModeState): ModeSummary
}
```

Mode Registration

Modes are registered at build time (built-in) or runtime (custom):

```
// Built-in mode registration
import { roundtable } from '@agora/mode-roundtable'
import { werewolf } from '@agora/mode-werewolf'

modeRegistry.register(roundtable)
modeRegistry.register(werewolf)

// Custom mode (future)
modeRegistry.registerCustom(userProvidedModeDefinition)
```

Mode Package Structure

```
packages/modes/roundtable/
├── index.ts           # Mode implementation
├── flow.ts            # RoundRobin flow controller config
├── schemas.ts         # Zod schemas for actions
├── prompts.ts         # System prompt templates
├── types.ts           # Mode-specific types
└── README.md
```

7. Phase 1: Roundtable Debate

Goal: Validate the platform core by shipping the simplest possible multi-agent interaction mode.

Timeline: 3-4 weeks

7.1 Overview

Multiple AI agents (each backed by a different LLM) debate a user-provided topic. The debate follows a structured format with opening statements, rebuttals, and closing arguments. A moderator agent manages the flow.

This is the ideal first mode because: - Single channel (no information isolation needed) - Simple flow (RoundRobin) - No game state beyond conversation history - High viral potential ("Watch Claude argue with GPT!") - Validates: Agent system, Room system, Flow control, Structured output, Event stream

7.2 User Stories

US-1.1: Create a Debate Room

> As a user, I want to create a debate room by providing a topic and selecting which AI models will participate, so that I can watch different LLMs argue with each other.

Acceptance Criteria: - [ ] User can enter a debate topic (free text, 10-500 characters) - [ ] User can add 2-8 agents to the debate - [ ] For each agent, user selects: name, model provider + model ID, short persona description (optional) - [ ] If no persona is provided, a default debater persona is generated - [ ] User can preview enriched personas before starting - [ ] Room configuration is persisted (survives page refresh)

US-1.2: Configure Debate Format

> As a user, I want to choose the debate format (number of rounds, speaking time, whether to include a moderator), so I can customize the experience.

Acceptance Criteria: - [ ] User can select number of rounds (1-5, default 3) - [ ] User can choose debate structure: `opening_only` | `opening_and_rebuttal` | `full` (opening + rebuttal + closing) - [ ] User can enable/disable a moderator agent (default: enabled) - [ ] When moderator is enabled, it uses a dedicated persona focused on fairness and time management - [ ] Configuration has sensible defaults so user can start with one click after adding agents

US-1.3: Watch the Debate

> As a user, I want to watch agents debate in real-time in a group-chat-style UI, so I can follow the conversation as it unfolds.

Acceptance Criteria: - [ ] Messages appear as chat bubbles with agent name, avatar, and model badge - [ ] Messages stream in real-time (token-by-token via SSE/Socket.io) - [ ] Each agent's messages are visually distinct (color-coded by agent) - [ ] Model provider badge shown on each message (e.g., "Claude", "GPT", "Gemini") - [ ] Moderator messages are visually distinct from debater messages - [ ] Current round and phase (opening / rebuttal / closing) is displayed - [ ] User can scroll up to see earlier messages without disrupting the live feed

US-1.4: Debate Results

> As a user, I want to see a summary of the debate when it ends, so I can understand the key arguments and positions.

Acceptance Criteria: - [ ] When debate ends, a summary is generated showing each agent's final position - [ ] Summary includes: topic, participants (with models), key arguments per agent, areas of agreement, areas of disagreement - [ ] User can share the debate summary via a link - [ ] User can view a replay of the full debate

US-1.5: Persona Enrichment

> As a user, I want to write a short character description and have the platform expand it into a full persona, so I don't have to write detailed prompts.

Acceptance Criteria: - [ ] User enters 1-3 sentences describing a character - [ ] Platform generates a detailed persona (personality, speech patterns, knowledge, behavioral tendencies) - [ ] User can review and edit the enriched persona - [ ] User can regenerate if unsatisfied - [ ] Enrichment takes less than 5 seconds

7.3 Technical Requirements

Debate Flow (RoundRobin)

```
For each round (1 to N):
  1. Moderator introduces the round
  2. For each debater (randomized order):
     a. Agent receives: topic, all previous messages, current round/phas
     b. Agent produces: structured DebateResponse (stance, argument, reb
     c. Message is posted to the public channel
  3. Moderator summarizes the round

After final round:
  4. Moderator delivers final summary
  5. Room status → 'completed'
```

Structured Output Schema

```
const DebateResponseSchema = z.object({
  stance: z.enum(['strongly_agree', 'agree', 'neutral', 'disagree', 'str
  argument: z.string().min(50).max(1000)
    .describe('Your argument for this round. Reference other participant
  rebuttal_to: z.string().optional()
    .describe('Name of the participant whose argument you are directly r
  concession: z.string().optional()
    .describe('Any point you concede from the opposing side'),
  confidence: z.number().min(0).max(1)
    .describe('How confident you are in your position (0 = not at all, 1
})
```

API Endpoints

Method	Path	Description
`POST`	`/api/rooms`	Create a new debate room
`GET`	`/api/rooms/:id`	Get room configuration and status
`PATCH`	`/api/rooms/:id`	Update room configuration
`POST`	`/api/rooms/:id/start`	Start the debate
`POST`	`/api/rooms/:id/pause`	Pause the debate

Method	Path	Description
`POST`	`/api/rooms/:id/resume`	Resume the debate
`GET`	`/api/rooms/:id/messages`	Get all messages (with pagination)
`GET`	`/api/rooms/:id/summary`	Get debate summary (after completion)
`POST`	`/api/agents/enrich-persona`	Enrich a short persona into a full character

Database Schema (Phase 1)

```sql
-- Rooms
CREATE TABLE rooms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  mode TEXT NOT NULL DEFAULT 'roundtable',
  mode_config JSONB NOT NULL DEFAULT '{}',
  status TEXT NOT NULL DEFAULT 'configuring',
  created_by TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  completed_at TIMESTAMPTZ
);

-- Agents (per room)
CREATE TABLE room_agents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  persona TEXT,
  enriched_persona TEXT,
  model_provider TEXT NOT NULL,
  model_id TEXT NOT NULL,
  role TEXT DEFAULT 'participant',
  status TEXT NOT NULL DEFAULT 'active',
  config JSONB NOT NULL DEFAULT '{}',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Messages
CREATE TABLE messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  channel_id UUID,
  agent_id UUID REFERENCES room_agents(id),
  type TEXT NOT NULL DEFAULT 'message',
  content TEXT NOT NULL,
  structured_data JSONB,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Events (for replay)
```

```
CREATE TABLE events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  type TEXT NOT NULL,
  channel_id UUID,
  agent_id UUID,
  payload JSONB NOT NULL,
  visibility TEXT NOT NULL DEFAULT 'public',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_messages_room_id ON messages(room_id);
CREATE INDEX idx_messages_created_at ON messages(created_at);
CREATE INDEX idx_events_room_id ON events(room_id);
CREATE INDEX idx_events_created_at ON events(created_at);
```

7.4 UI Requirements (Phase 1)

- **Home Page**: List of active/recent rooms, "Create Room" button
- **Room Setup Page**: Topic input, agent configuration cards, format settings, "Start Debate" button
- **Room View (Chat)**: Group-chat-style message feed with streaming, round/phase indicator, participant sidebar
- **Summary View**: Post-debate summary with key arguments, positions, and share button

UI is functional, not polished. Phase 3 handles UX polish.

7.5 Phase 1 Acceptance Criteria

- [ ] A user can create a room, add 2-6 agents with different models, and start a roundtable debate
- [ ] Agents debate coherently, maintaining their personas and responding to each other's arguments
- [ ] Messages stream in real-time in a chat-style UI
- [ ] Each agent's model provider is visible
- [ ] The debate follows the configured round structure

- [ ] A summary is generated when the debate ends
- [ ] The full session can be replayed
- [ ] Response latency per agent turn is under 15 seconds (for a typical response)
- [ ] The system handles at least 10 concurrent rooms

8. Phase 2: Werewolf (狼人杀)

Goal: Introduce Channel-based information isolation and StateMachine flow control by implementing the classic social deduction game.

Timeline: 4-5 weeks (after Phase 1)

8.1 Overview

Werewolf is a social deduction game where a minority of "wolves" try to eliminate "villagers" while the village tries to identify and vote out the wolves. This mode validates: - **Channel system**: Wolves have a private channel; dead players move to a spectator channel - **StateMachine flow**: Night → Day Discussion → Day Vote → Elimination - **Structured decisions**: Kill targets, vote targets, special ability usage - **Role-based information asymmetry**: Each role knows different things

8.2 Game Configuration

```
const WerewolfConfigSchema = z.object({
  playerCount: z.number().min(6).max(12),
  roles: z.object({
    werewolves: z.number().min(1).max(4),
    seer: z.boolean().default(true),        // Can check one player per
    doctor: z.boolean().default(true),       // Can protect one player p
    hunter: z.boolean().default(false),      // Takes someone with them
    witch: z.boolean().default(false),       // Has one save potion and
  }),
  dayDiscussionRounds: z.number().min(1).max(3).default(2),
  enableLastWords: z.boolean().default(true),  // Eliminated player gets
  moderatorModel: z.string().default('claude-sonnet-4-20250514'),
})
```

8.3 User Stories

US-2.1: Create a Werewolf Game

> As a user, I want to create a werewolf game by selecting the number of players and which special roles to include, so I can customize the game setup.

Acceptance Criteria: - [ ] User selects player count (6-12) - [ ] User enables/disables special roles (seer, doctor, hunter, witch) - [ ] System validates role configuration (e.g., werewolves < half of players) - [ ] User adds agents with personas (or uses default names/personas) - [ ] Roles are randomly assigned and kept secret

US-2.2: Night Phase

> As a spectator, I want to watch the night phase where wolves choose a target and special roles use their abilities, seeing everything in god mode.

Acceptance Criteria: - [ ] Wolves discuss and vote on a kill target in a private wolf channel - [ ] Seer selects a player to investigate (structured output: player ID) - [ ] Seer receives the result (wolf or villager) privately - [ ] Doctor selects a player to protect (structured output: player ID) - [ ] If doctor protects the wolf target, the kill is prevented - [ ] Night actions happen in parallel (wolves, seer, doctor all act simultaneously) - [ ] Spectators in god mode see all night actions; regular spectators see "Night falls..."

US-2.3: Day Discussion

> As a spectator, I want to watch agents discuss suspicions during the day phase, seeing them accuse, defend, and strategize in the public channel.

Acceptance Criteria: - [ ] All living agents participate in day discussion via the public channel - [ ] Moderator announces night results ("Player X was eliminated" or "Nobody died") - [ ] Discussion follows RoundRobin within the day phase (each agent speaks once per round) - [ ] Agents reference previous behavior, voting patterns, and night results - [ ] Wolves attempt to blend in and deflect suspicion - [ ] Discussion respects the configured number of rounds

US-2.4: Voting and Elimination

> As a spectator, I want to watch agents vote to eliminate a suspect, with each agent explaining their vote.

Acceptance Criteria: - [ ] After discussion, each living agent votes (structured output: target player ID + reasoning) - [ ] Votes are revealed simultaneously (not sequentially) - [ ] Player with the most votes is eliminated - [ ] In case of a tie, a revote occurs between tied players (max 1 revote, then no elimination) - [ ] Eliminated player is moved to spectator channel - [ ] If `enableLastWords`, eliminated player gives a final speech before being removed

US-2.5: Game End

> As a spectator, I want to see the game end when either all wolves are eliminated or wolves equal villagers, with a full reveal of roles and key moments.

Acceptance Criteria: - [ ] Game ends when: all wolves eliminated (village wins) or wolves >= villagers (wolves win) - [ ] Final summary reveals all roles - [ ] Summary highlights key moments (correct seer checks, clutch doctor saves, successful wolf deceptions) - [ ] Win/loss attribution per agent - [ ] Full game replay available with channel filtering

8.4 State Machine

```
┌──────────────┐
│  NIGHT       │
│  - Wolf kill │──────────────┐
│  - Seer check│              │
│  - Doctor    │              ▼
│    protect   │      ┌──────────────────┐
└──────────────┘      │  DAWN            │
       ▲              │  - Resolve       │
       │              │    night         │
       │              │  - Announce      │
       │              │    results       │
       │              └──────────────────┘
       │                      │
       │                      ▼
       │              ┌──────────────────┐
       │              │  DAY             │
       │              │  - Discussion    │
       │              │    (N rounds)    │
       │              └──────────────────┘
       │                      │
       │                      ▼
       │              ┌──────────────────┐
       │              │  VOTE            │
       │              │  - Each player   │
       │              │    votes         │
       │              │  - Resolve tie   │
       │              └──────────────────┘
       │                      │
       │                      ▼
       │              ┌──────────────────┐      ┌──────────────────┐
       │              │  ELIMINATION     │──────►│  GAME OVER       │
       │              │  - Remove        │      │  (if win         │
       │              │    player        │      │   condition)     │
       │              │  - Last words    │      └──────────────────┘
       └──────────────└──────────────────┘
```

8.5 Channel Configuration

Channel	Type	Members	Description
`town-square`	public	All living agents	Day discussion, voting, announcements
`wolf-den`	private	Wolves only	Night discussion, kill target selection
`seer-vision`	whisper	Seer + System	Seer investigation results
`doctor-ward`	whisper	Doctor + System	Doctor protection target
`graveyard`	spectator	Eliminated agents	Dead players watch but can't interact
`god-view`	spectator	God-mode observers	Sees everything including agent reasoning

8.6 Structured Output Schemas

```
const WolfKillSchema = z.object({
  action: z.literal('kill'),
  target: z.string().describe('Agent ID of the player to kill'),
  reasoning: z.string().describe('Why this target? (visible only to wolv
})

const SeerCheckSchema = z.object({
  action: z.literal('investigate'),
  target: z.string().describe('Agent ID of the player to investigate')
})

const DoctorProtectSchema = z.object({
  action: z.literal('protect'),
  target: z.string().describe('Agent ID of the player to protect (can be
})

const DayVoteSchema = z.object({
  action: z.literal('vote'),
  target: z.string().describe('Agent ID of the player to vote for elimir
  reasoning: z.string().describe('Public explanation for your vote'),
  confidence: z.number().min(0).max(1)
})
```

8.7 Phase 2 Acceptance Criteria

- [ ] A werewolf game can be configured with 6-12 AI agents and customizable roles
- [ ] Wolves discuss privately and villagers cannot see wolf channel messages
- [ ] Seer receives accurate investigation results privately
- [ ] Doctor protection prevents wolf kills when targeting the same player
- [ ] Day discussion is coherent, with agents forming suspicions based on available information
- [ ] Voting produces valid eliminations with tie-breaking
- [ ] Game ends correctly when win conditions are met
- [ ] The Channel system prevents any information leakage between channels

- [ ] Full game replay with channel filtering works
- [ ] God mode shows all channels and agent reasoning simultaneously

9. Phase 3: UX Polish

Goal: Transform the functional prototype into a visually compelling, content-worthy experience.

Timeline: 3-4 weeks (after Phase 2)

9.1 Room View Visualization

The Room View is the centerpiece of the spectator experience.

Requirements: - [ ] Animated message bubbles with typing indicators - [ ] Agent avatars (AI-generated or user-uploaded) - [ ] Model provider badges (Claude logo, GPT logo, Gemini logo, etc.) - [ ] @mention highlighting in messages - [ ] Phase/round progress indicator - [ ] Sound effects for key events (optional, user-configurable) - [ ] Dark mode support - [ ] Responsive layout (desktop primary, tablet secondary)

9.2 Persona Editor

A visual tool for creating and customizing agent personas.

Requirements: - [ ] Rich text editor for persona descriptions - [ ] Side-by-side view: user's short description → enriched persona - [ ] Personality trait sliders (creativity, aggressiveness, humor, formality) - [ ] Speech pattern examples (generated from persona) - [ ] "Test this persona" — send a sample prompt and see how the agent responds - [ ] Save/load persona templates - [ ] Persona library with pre-built characters

9.3 Human Players

Allow humans to participate alongside AI agents.

Requirements: - [ ] Room creator can mark participant slots as "human" or "AI" - [ ] Human players join via invite link - [ ] Human players see only their permitted channels (same isolation as AI agents) - [ ] Human players type responses manually (with optional time limits) - [ ] AI agents and human players are indistinguishable to other agents (Turing test mode, optional) - [ ] Human players can use "suggest" to get AI-generated response suggestions - [ ] Works in all modes (debate, werewolf, etc.)

9.4 Enhanced Spectator Experience

Requirements: - [ ] Spectator count displayed on active rooms - [ ] Live reactions (emoji reactions on messages) - [ ] Spectator chat (separate from agent conversation) - [ ] Picture-in-picture mode for watching multiple channels simultaneously - [ ] Shareable room URLs with embed support - [ ] Screenshot/clip generation for sharing - [ ] Streamlined replay UI with playback speed control

9.5 Phase 3 Acceptance Criteria

- [ ] Room View is visually polished with smooth animations and clear visual hierarchy
- [ ] Persona editor allows creation of detailed characters with preview
- [ ] At least one human can join a room and participate alongside AI agents
- [ ] Spectator mode shows real-time view with reactions and chat
- [ ] The UI is responsive on desktop and tablet
- [ ] A first-time user can create and start a room in under 2 minutes

10. Phase 4: Script Kill (剧本杀)

Goal: Implement a narrative-driven murder mystery mode that introduces clue distribution, branching storylines, and long-term memory.

Timeline: 5-6 weeks (after Phase 3)

10.1 Overview

Script Kill (剧本杀) is a murder mystery party game popular in China. Players each receive a character script with background, relationships, secrets, and clues. Through multiple rounds of investigation and discussion, they try to solve the murder while protecting their own secrets. This mode validates: - **Complex channel configurations**: Each player has unique private information - **Clue distribution system**: Timed release of clues across investigation rounds - **Long-term memory**: Agents must remember and cross-reference clues across rounds - **Narrative generation**: AI GM creates coherent story arcs

10.2 Script System

Script Structure

```
interface Script {
  id: string
  title: string
  synopsis: string                    // Public premise
  setting: ScriptSetting              // Time period, location, atmosphe
  characters: ScriptCharacter[]       // Character templates
  clues: Clue[]                       // All clues in the game
  timeline: TimelineEvent[]           // The "true" sequence of events
  investigation_rounds: number        // How many rounds of clue gatheri
  solution: Solution                  // The truth (for GM reference)
}

interface ScriptCharacter {
  id: string
  name: string
  publicProfile: string               // What everyone knows about this
  privateBackground: string           // Known only to this player
  secrets: Secret[]                   // Things this character is hiding
  relationships: CharacterRelationship[]
  personalGoal: string                // What this character wants (beyo
  initialClues: string[]              // Clue IDs this character starts
}

interface Clue {
  id: string
  content: string
  type: 'physical' | 'testimonial' | 'documentary' | 'environmental'
  location: string                     // Where this clue is found
  availableInRound: number             // When this clue becomes availab
  requiredToSolve: boolean             // Is this clue essential?
  linkedClues: string[]                // Other clue IDs that connect to
  visibleTo: string[] | 'all'          // Which characters can find this
}
```

Script Generation

The platform supports two script sources: 1. **Pre-built scripts**: Curated scripts included with the platform 2. **AI-generated scripts**: User provides a theme/setting, and an AI (using a high-capability model) generates a complete script

AI script generation flow: 1. User provides: theme, number of players, tone (serious/comedic), setting preferences 2. AI generates: full script with characters, timeline, clues, and solution 3. Human reviews and approves (or requests regeneration)

10.3 User Stories

US-4.1: Start a Script Kill Game

> As a user, I want to select a murder mystery script and assign AI agents to characters, watching them investigate and accuse each other.

Acceptance Criteria: - [ ] User selects from available scripts or generates a new one - [ ] User assigns agents to characters (or auto-assigns) - [ ] Each agent receives its character's private background and initial clues - [ ] GM agent (dedicated model) manages the game - [ ] Game starts with the GM setting the scene

US-4.2: Investigation Rounds

> As a spectator, I want to watch agents investigate the crime scene, discovering clues and questioning each other.

Acceptance Criteria: - [ ] Each investigation round, agents choose locations to investigate - [ ] Agents receive clues based on their location choice and the round's available clues - [ ] Agents can question other agents (who may lie or deflect based on their secrets) - [ ] Clues discovered by an agent are added to their private knowledge - [ ] GM narrates clue discoveries with atmospheric descriptions - [ ] Spectators can see all clues (god mode) or only publicly shared information

US-4.3: Discussion and Accusation

> As a spectator, I want to watch agents share (or withhold) information, form theories, and accuse suspects.

Acceptance Criteria: - [ ] After each investigation round, agents discuss in the public channel - [ ] Agents strategically share, withhold, or misrepresent their clues - [ ] Agents form and articulate theories about the murder - [ ] Agents can directly accuse others (structured: accusation + evidence) - [ ] Accused agents must respond (deny, deflect, counter-accuse) - [ ] GM tracks the state of accusations and evidence

US-4.4: Resolution

> As a spectator, I want to see the final vote and reveal, learning whether the agents solved the mystery correctly.

Acceptance Criteria: - [ ] Final vote: each agent votes for who they believe is the murderer (structured output) - [ ] GM reveals the true answer and walks through the timeline - [ ] Summary shows: who voted correctly, which clues were found/missed, which secrets were exposed - [ ] Post-game analysis: how close were agents to the truth? What misled them?

10.4 Channel Configuration

Channel	Type	Members	Description
`main-hall`	public	All characters + GM	Public discussion, accusations, GM narration
`{character}-private`	private	Individual character	Private clues, internal reasoning, personal goals
`{character}-{character}`	whisper	Two characters	Private conversations between characters
`investigation-{location}`	private	Characters at location	Clue discovery at specific locations
`gm-channel`	broadcast	GM → All	GM announcements, scene setting, phase transitions

Channel	Type	Members	Description
`god-view`	spectator	Observers	Full visibility into all channels

10.5 Phase 4 Acceptance Criteria

- [ ] A script kill game runs with 4-8 AI agents and a GM agent
- [ ] Each agent has unique private information that is never leaked to other agents
- [ ] Clues are distributed correctly based on location choices and round availability
- [ ] Agents form coherent theories based on accumulated evidence
- [ ] Agents strategically share and withhold information consistent with their character goals
- [ ] Final vote resolves the mystery with a full reveal
- [ ] At least 3 pre-built scripts are available
- [ ] AI-generated scripts produce playable mysteries (tested with at least 5 generated scripts)

11. Phase 5: TRPG (跑团)

Goal: Implement tabletop RPG support with an AI Game Master, dice mechanics, character progression, and emergent narrative.

Timeline: 6-8 weeks (after Phase 4)

11.1 Overview

TRPG (Tabletop Role-Playing Game) is the most complex mode, requiring: - **AI Game Master**: Manages world state, narrates events, adjudicates rules - **Dice mechanics**: Skill checks, combat rolls, saving throws - **Character sheets**: Stats, skills, inventory, health - **Persistent campaigns**: Multi-session storylines with character progression - **Emergent narrative**: Story adapts to player decisions

11.2 Game Master Agent

The GM is a specialized agent with elevated capabilities:

```
interface GMAgent extends Agent {
  /** Generate the next scene/encounter based on current story state */
  narrate(worldState: WorldState, recentActions: AgentAction[]): Promise

  /** Adjudicate a player action — determine if a dice roll is needed ar
  adjudicate(action: TRPGAction, character: CharacterSheet): Promise<Adj

  /** Describe the outcome of a dice roll */
  resolveRoll(roll: DiceRoll, adjudication: Adjudication): Promise<Narra

  /** Manage NPCs — decide their actions, dialogue, and reactions */
  controlNPC(npc: NPC, context: SceneContext): Promise<NPCAction>

  /** Advance the story — introduce plot points, complications, resoluti
  advanceStory(worldState: WorldState): Promise<StoryBeat>
}
```

11.3 Dice System

```
interface DiceRoll {
  notation: string        // e.g., "2d6+3", "1d20", "4d6kh3" (keep highe
  results: number[]       // Individual die results
  modifier: number        // Added modifier
  total: number           // Final result
  type: RollType          // 'skill_check' | 'attack' | 'damage' | 'savi
  dc?: number             // Difficulty class (if applicable)
  success?: boolean       // Whether the roll met/exceeded the DC
}
```

Dice rolls are transparent and verifiable: - All rolls use server-side RNG (not AI-generated numbers) - Roll results are displayed to all players (unless GM marks as secret) - Roll history is persisted for the session

11.4 Character System

```
interface CharacterSheet {
  id: string
  name: string
  class: string
  level: number
  stats: {
    strength: number
    dexterity: number
    constitution: number
    intelligence: number
    wisdom: number
    charisma: number
  }
  hitPoints: { current: number; max: number }
  skills: Skill[]
  inventory: Item[]
  backstory: string
  personalityTraits: string[]
  bonds: string[]
  flaws: string[]
  experience: number
}
```

Character creation supports: - Manual stat assignment - Random stat generation (4d6 drop lowest) - AI-assisted backstory generation based on class and traits - Integration with persona enrichment system

11.5 User Stories

US-5.1: Create a TRPG Campaign

> As a user, I want to set up a TRPG campaign with a setting, AI GM, and player characters, so we can play an ongoing adventure.

Acceptance Criteria: - [ ] User provides campaign setting (genre, world description, tone) - [ ] AI GM generates the opening scenario based on the setting - [ ] User creates or auto-generates character sheets for player agents - [ ] Campaign state persists across multiple sessions - [ ] User can configure rules complexity (simple, standard, advanced)

US-5.2: Exploration and Roleplay

> As a spectator, I want to watch agents explore the world, interact with NPCs, and roleplay their characters.

Acceptance Criteria: - [ ] GM describes scenes with environmental details, NPCs, and points of interest - [ ] Player agents decide actions in character (structured: action_type, description, target) - [ ] GM adjudicates actions, calling for dice rolls when outcome is uncertain - [ ] NPCs are voiced by the GM with distinct personalities - [ ] Players can have private conversations (whisper channels) - [ ] World state updates based on player actions

US-5.3: Combat

> As a spectator, I want to watch turn-based combat with dice rolls, tactical decisions, and dramatic narration.

Acceptance Criteria: - [ ] Combat uses initiative-based turn order (rolled at combat start) - [ ] Each combatant's turn: move, action, bonus action (structured output) - [ ] Attack rolls and damage rolls are transparent - [ ] GM narrates combat outcomes with dramatic flair - [ ] Health/status tracking for all combatants - [ ] Combat ends when enemies are defeated, players flee, or players are defeated - [ ] GM adapts difficulty dynamically (within reason) to maintain fun

US-5.4: Campaign Progression

> As a user, I want campaigns to persist across sessions, with character progression, story continuity, and growing consequences.

Acceptance Criteria: - [ ] Character sheets persist: HP, inventory, experience, level - [ ] Story state persists: plot progress, NPC relationships, world changes - [ ] GM summarizes previous session at the start of each new session - [ ] Long-term memory tracks key story events and character arcs - [ ] Characters can level up between sessions

11.6 Phase 5 Acceptance Criteria

- [ ] AI GM runs a coherent TRPG session with 2-6 player agents
- [ ] Dice mechanics work correctly with transparent, server-side rolls
- [ ] Character sheets track stats, HP, inventory, and experience
- [ ] Combat follows turn-based rules with proper resolution
- [ ] GM adapts narrative to player decisions (not railroad)
- [ ] Campaign state persists across at least 3 sessions
- [ ] At least 3 campaign settings are available (fantasy, sci-fi, modern)
- [ ] GM maintains consistent world state and NPC behavior

12. Phase 6: Platform

Goal: Transform Agora from a product into a platform by enabling custom modes, sharing, and advanced orchestration.

Timeline: 8-12 weeks (after Phase 5)

12.1 Custom Mode SDK

Allow developers to create and publish their own interaction modes.

Requirements: - [ ] Published `@agora/mode-sdk` npm package with the Mode interface, typed helpers, and documentation - [ ] Mode template generator (`npx create-agora-mode`) - [ ] Local development server for testing custom modes - [ ] Mode validation (schema checks, simulation tests) - [ ] Mode publishing to the Agora registry

12.2 Agent Marketplace

A community-driven marketplace for sharing agent personas.

Requirements: - [ ] Users can publish personas (short description + enriched persona + recommended models) - [ ] Browse and search personas by category, personality type, skill domain - [ ] Rating and review system - [ ] "Fork" a persona to customize it - [ ] Featured/trending personas on the home page - [ ] Pre-built teams (e.g., "Philosophy Roundtable" with 5 philosopher personas)

12.3 Replay System (Enhanced)

Requirements: - [ ] Public replay gallery (opt-in from room creators) - [ ] Embeddable replays (iframe embed for blogs, social media) - [ ] Replay analytics: message count, token usage per agent, key decision points - [ ] Annotated replays: spectators can add comments at specific timestamps - [ ] Highlight reel generation: AI-generated clips of the most interesting moments

12.4 Hierarchical Flow Controller

Support complex multi-agent workflows beyond games.

Requirements: - [ ] Manager-worker agent relationships - [ ] Task delegation: manager assigns tasks to workers, workers report back - [ ] Aggregation: manager synthesizes worker outputs - [ ] Escalation: workers can escalate to manager when stuck - [ ] Useful for: company simulations, research teams, project management scenarios

12.5 Phase 6 Acceptance Criteria

- [ ] A developer can create, test, and publish a custom mode using the SDK
- [ ] At least 2 community-created modes are published and playable

- [ ] Agent marketplace has browse, search, publish, and fork functionality
- [ ] Replay gallery shows at least 50 public replays
- [ ] Hierarchical flow controller supports at least one demo scenario (e.g., startup simulation)
- [ ] Platform supports 100+ concurrent rooms

13. Non-Functional Requirements

13.1 Performance

Metric	Target	Notes
Agent response latency (p50)	< 5 seconds	Measured from turn start to first token
Agent response latency (p95)	< 15 seconds	Model-dependent; slower for Opus-class models
Message delivery latency	< 200ms	Socket.io broadcast to all room participants
Room creation time	< 2 seconds	Including database write
Persona enrichment time	< 5 seconds	Single LLM call
UI time to interactive	< 3 seconds	First meaningful paint on room page
Concurrent rooms (Phase 1)	10+	Single Vercel deployment

Metric	Target	Notes
Concurrent rooms (Phase 6)	100+	With horizontal scaling

13.2 Scalability

- **Stateless API layer**: Next.js API routes on Vercel (auto-scales)
- **Socket.io**: Single server initially; Redis adapter for multi-server scaling
- **Database**: Supabase Postgres (connection pooling via Supavisor)
- **LLM calls**: Rate limiting per model provider; queue system for burst traffic
- **Event storage**: Time-series partitioning for events table at scale

13.3 Reliability

- **Graceful degradation**: If one model provider is down, room continues with remaining agents (affected agent is marked as unavailable)
- **Auto-retry**: Failed LLM calls retry up to 3 times with exponential backoff
- **Session recovery**: If server restarts mid-session, room state is recoverable from database
- **Data durability**: All messages and events are persisted before being broadcast

13.4 Security

Concern	Mitigation
API key exposure	All LLM calls happen server-side; API keys stored in environment variables, never sent to client
Prompt injection	Agent system prompts include injection resistance; structured output constrains agent actions
Information leakage	Channel isolation enforced at platform level; agents never receive messages from channels they don't belong to

Concern	Mitigation
Abuse	Rate limiting on room creation and agent turns; content moderation on public rooms
Data privacy	User data stored in Supabase with RLS; rooms can be private; replay sharing is opt-in
Auth	Supabase Auth with OAuth (GitHub, Google); anonymous access for spectating public rooms

13.5 Observability

- **Logging**: Structured JSON logs for all API routes and LLM calls
- **Metrics**: Token usage per agent, response latency per model, active rooms, error rates
- **Tracing**: Request tracing from UI action → API → LLM → response
- **Alerts**: Notify on high error rates, model provider outages, or unusual token consumption

13.6 Accessibility

- Keyboard navigation for all UI elements
- Screen reader compatibility for the chat interface
- Sufficient color contrast (WCAG AA)
- Reduced motion option for animations

14. Success Metrics

Phase 1: Roundtable Debate

Metric	Target	Measurement
GitHub stars	500+	GitHub API
Completed debates	100+	Database query
Unique users (creators)	50+	Supabase Auth
Average debate completion rate	> 80%	Started rooms that reach completion
Social shares	20+	Share link clicks / embeds
Average session duration	> 5 minutes	Time from room start to completion
Developer PRs	5+	GitHub (validates open-source traction)

Phase 2: Werewolf

Metric	Target	Measurement
Completed games	200+	Database query
Channel isolation: zero leaks	0 violations	Automated test suite + audit logs
Game coherence rating	> 4/5	User feedback (post-game survey)
Average game duration	10-30 minutes	Time from start to completion
Returning users	> 30%	Users who create 2+ rooms

Phase 3: UX Polish

Metric	Target	Measurement
Room creation time	< 2 minutes	Time from landing to room start
Human-AI games	20+	Rooms with at least 1 human participant
Spectator engagement	> 3 min avg watch time	Socket.io connection duration
Mobile/tablet visits	> 15% of traffic	Analytics

Phase 4: Script Kill

Metric	Target	Measurement
Mystery solve rate	30-70%	Correct final votes (too high = too easy, too low = too hard)
Generated scripts playability	> 80% viable	Playtesting with AI agents
Average session duration	20-45 minutes	Database

Phase 5: TRPG

Metric	Target	Measurement
Multi-session campaigns	10+	Campaigns with 3+ sessions
Narrative coherence	> 4/5	User feedback

Metric	Target	Measurement
Dice roll fairness	Within statistical norms	Chi-squared test on roll distribution

Phase 6: Platform

Metric	Target	Measurement
Published custom modes	5+	Mode registry
Published personas	50+	Agent marketplace
Third-party mode sessions	100+	Rooms using non-built-in modes
GitHub stars	5,000+	GitHub API

15. Open Questions & Risks

Open Questions

#	Question	Impact	Owner	Status
Q1	**Cost model**: Who pays for LLM API calls? User brings their own keys? Platform subsidizes? Freemium with token limits?	Determines business model and user onboarding friction	Product	Open
Q2	**Latency vs. quality tradeoff**: Should we default to faster models (GPT-4o-mini, Gemini	Affects UX (waiting 15s per turn in a	Engineering	Open

#	Question	Impact	Owner	Status
	Flash) and let users opt into slower, higher-quality models? Or always use frontier models?	6-player werewolf game = long waits)		
Q3	**Content moderation**: How do we handle agents generating harmful, offensive, or NSFW content? Especially in freeform and custom modes?	Legal liability, platform trust	Product + Legal	Open
Q4	**Human participation UX**: When humans play alongside AI, should there be time limits per turn? Should humans be identifiable as human or anonymous?	Affects game balance and social dynamics	Product	Open
Q5	**Script Kill script IP**: For pre-built scripts, do we create original content or adapt public domain stories? How do AI-generated scripts avoid reproducing copyrighted plots?	Legal risk	Legal	Open
Q6	**Internationalization**: Platform UI in English first? When do we add Chinese (primary market for script kill, significant AI enthusiast community)?	Market size	Product	Open — lean toward bilingual from Phase 1

#	Question	Impact	Owner	Status
Q7	**Agent personality drift**: Over long sessions (especially TRPG campaigns), agents may drift from their persona. How do we detect and correct this?	Quality of experience	Engineering	Open
Q8	**Replay privacy**: If a room is private but produces a great moment, can the creator make just a clip public? What about other participants' consent?	Privacy, trust	Product	Open

Risks

#	Risk	Likelihood	Impact	Mitigation
R1	**LLM costs are prohibitive** — A 12-agent werewolf game with 5 rounds could cost $5-15 in API calls	High	High	Default to cost-efficient models (Sonnet, GPT-4o-mini); BYOK (bring your own key) as first option; batch optimization; prompt caching
R2	**Agent quality inconsistency across models** — Cheaper/weaker models may break character, produce incoherent arguments,	High	Medium	Model-specific prompt tuning; fallback to stronger model on structured output failure; model recommendations per mode

#	Risk	Likelihood	Impact	Mitigation
	or fail structured output			
R3	**Information isolation bugs** — A single channel leak in werewolf ruins the game	Medium	Critical	Extensive test suite for channel isolation; property-based testing; audit logging; isolation enforced at data access layer, not prompt level
R4	**"Empty room" problem** — Nobody discovers the platform; rooms are created but not shared	Medium	High	Focus viral format (debate clips) first; built-in sharing; embed support; content creator outreach
R5	**Scope creep from game complexity** — Script Kill and TRPG are significantly more complex than roundtable; timeline slips	High	Medium	Strict phase boundaries; ship each phase independently; resist adding features from later phases early
R6	**Model provider rate limits or outages** — Multiple concurrent rooms hitting the same provider	Medium	Medium	Multi-provider support from day 1; graceful degradation; queue system with backpressure
R7	**Open source but no community** — Project is open source but fails to attract contributors	Medium	Medium	Excellent documentation; "good first issue" labels; Mode SDK designed for external contributors;

#	Risk	Likelihood	Impact	Mitigation
				active Discord/community
R8	**Context window limits** — Long werewolf games or TRPG campaigns exceed model context windows	Medium	Medium	Sliding window with summarization; session memory system; aggressive prompt optimization

Dependencies

Dependency	Type	Risk	Notes
Vercel AI SDK	Technical	Low	Well-maintained, actively developed, broad model support
Supabase	Infrastructure	Low	Managed Postgres, auth, realtime — reduces ops burden
LLM API availability	External	Medium	Claude, GPT, Gemini, Qwen — need at least 2 providers available
Socket.io	Technical	Low	Mature library; Redis adapter available for scaling
Next.js 15	Technical	Low	Stable release, well-documented

Appendix A: Glossary

Term	Definition
Agent	An AI participant in a room, backed by an LLM with a persona
Room	A container for a multi-agent interaction session
Channel	A communication pathway with defined membership and visibility rules
Mode	A pluggable interaction pattern (roundtable, werewolf, script kill, TRPG, custom)
Flow Controller	The component that determines who acts when
Persona	The character description that shapes an agent's behavior
Enrichment	The process of expanding a short persona into a detailed character
Structured Output	LLM output constrained by a Zod schema to produce verifiable data
God Mode	Observation mode that reveals all information including private channels and agent reasoning
BYOK	Bring Your Own Key — users provide their own LLM API keys
GM	Game Master — the agent that manages world state and narrates in TRPG mode

Appendix B: Competitive Landscape

Project	What It Does	Why Agora Is Different
AgentScope (Alibaba)	Multi-agent framework (Python library)	Agora is a user-facing platform, not a developer framework. Built for watching/participating, not just orchestrating
AutoGen (Microsoft)	Multi-agent conversation framework	Developer tool for building agents; no game/entertainment focus; no spectator UX
CrewAI	Multi-agent task orchestration	Focused on productive workflows, not interactive entertainment; no real-time observation
ChatArena	LLM debate/game arena	Research-focused; no production UI; limited mode support; appears unmaintained
Various werewolf bots	Single-game implementations	Single-mode, not a platform; no multi-model support; no spectator experience

Agora's differentiation: **platform** (not framework), **multi-mode** (not single-game), **multi-model** (not single-provider), **spectator-first UX** (not developer-only), **open source** (not closed).

Appendix C: Reference Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                          Frontend                                 │
│  Next.js 15 (App Router) + Tailwind + shadcn/ui                   │
│   ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐            │
│   │ Home     │ │ Room     │ │ Room     │ │ Persona  │            │
│   │ Page     │ │ Setup    │ │ View     │ │ Editor   │            │
│   └──────────┘ └──────────┘ └──────────┘ └──────────┘            │
│                       │ Socket.io │ REST                          │
└───────────────────────┼──────────┼──────────────────────────────┘
                        │          │
┌───────────────────────┼──────────┼──────────────────────────────┐
│                     API Layer                                     │
│  Next.js API Routes + Socket.io Server                            │
│   ┌──────────────────────────────────────────────────────┐        │
│   │ Room API │ Agent API │ Message API │ Mode API         │        │
│   └──────────────────────────────────────────────────────┘        │
└───────────────────────┼──────────────────────────────────────────┘
                        │
┌───────────────────────┼──────────────────────────────────────────┐
│                  Platform Core (packages/core)                    │
│   ┌────────┐ ┌────────┐ ┌────────┐ ┌────────┐ ┌────────┐         │
│   │ Agent  │ │ Room   │ │Channel │ │ Flow   │ │Memory  │         │
│   │ System │ │ System │ │ System │ │Control │ │ System │         │
│   └────────┘ └────────┘ └────────┘ └────────┘ └────────┘         │
│   ┌────────────────┐ ┌────────────────┐ ┌────────────────┐        │
│   │ Structured     │ │ Event Bus      │ │ Observation    │        │
│   │ Output (Zod)   │ │                │ │ Layer          │        │
│   └────────────────┘ └────────────────┘ └────────────────┘        │
└───────────────────────┼──────────────────────────────────────────┘
                        │
┌───────────────────────┼──────────────────────────────────────────┐
│                  Mode Layer (packages/modes)                      │
│   ┌──────────────┐ ┌──────────────┐ ┌──────────────┐             │
│   │ Roundtable   │ │ Werewolf     │ │ Script       │             │
│   │ Debate       │ │ 狼人杀       │ │ Kill 剧本杀  │             │
│   └──────────────┘ └──────────────┘ └──────────────┘             │
│   ┌──────────────┐ ┌──────────────┐                               │
│   │ TRPG         │ │ Custom       │                               │
│   │ 跑团         │ │ Modes        │                               │
│   └──────────────┘ └──────────────┘                               │
└───────────────────────┼──────────────────────────────────────────┘
                        │
```

```
┌──────────────────────┬──────────────────────────────────────────┐
│                  Infrastructure                                  │
│  ┌──────────────┐ ┌──────────────┐ ┌──────────────┐             │
│  │ Vercel AI    │ │ Supabase     │ │ Socket.io    │             │
│  │ SDK          │ │ (Postgres)   │ │ (Realtime)   │             │
│  └──────────────┘ └──────────────┘ └──────────────┘             │
│  ┌──────────────┐ ┌──────────────┐                              │
│  │ Vercel       │ │ Redis        │                              │
│  │ (Deploy)     │ │ (Optional)   │                              │
│  └──────────────┘ └──────────────┘                              │
└──────────────────────────────────────────────────────────────────┘
```